

Reviewer Pack — Minimal Order of Quadratic Forms and Communication Complexit...

Entry e4cad27d697b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 23:47:25 UTC

Minimal Order of Quadratic Forms and Communication Complexity Rank Variance

Entry ID: e4cad27d697b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 23:47:25 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quadratic Form Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every k -CNF formula φ with n variables, the minimal order of a non-degenerate quadratic form associated with φ is linearly correlated with the variance of the communication complexity rank of all its monotone duals, such that $\text{MinimalOrder}(\varphi) = \Theta(\text{VarianceRankMonotoneDuals}(\varphi))$.

Rationale (proposer's reasoning):

Quadratic forms provide a rich algebraic structure that can be used to encode boolean functions, potentially revealing hidden complexities in communication protocols. The variance of the communication complexity rank across different monotone duals could shed light on the inherent complexity of the original formula.

Taxonomy category: QuadraticFormCommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9d669e68d5a7c889

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all k -CNF formulas with n variables ($n \leq 40$), the correlation coefficient between $\text{MinimalOrder}(\varphi)$ and $\text{VarianceRankMonotoneDuals}(\varphi)$ is ≥ 0.8 , AND the mean difference between $\text{MinimalOrder}(\varphi)$ and $\text{VarianceRankMonotoneDuals}(\varphi)$ across 30 seeds is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quadratic form theory" AND "communication complexity rank" - "minimal order quadratic forms" AND "monotone duals" - "variance of communication complexity rank" AND "minimal order"

Top relevant hits considered: - [s2:10.1145/3583668.3594596] On the Amortized Communication Complexity of Byzantine Broadcast - [s2:10.1109/SFCS.1995.492457] Spectral methods for matrix rigidity with applications to size-depth tradeoffs and communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_k_cnf(n, k):
    clauses = []
    for _ in range(k):
        clause = set()
        while len(clause) < 3:
            var = random.randint(1, n)
            if var not in clause:
                clause.add(var)
        clauses.append(list(clause))
    return clauses

def k_cnf_to_quadratic_form(clauses):
    n = max(max(clause) for clause in clauses)
    A = [[0] * n for _ in range(n)]
    for clause in clauses:
        for c in clause:
            A[c-1][c-1] += 1
            for d in clause:
                if c != d:
                    A[c-1][d-1] -= Fraction(1, len(clause))
                    A[d-1][c-1] -= Fraction(1, len(clause))
    return A

def determinant(A):
    n = len(A)
    if n == 1:
```

```

        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def minimal_order_quadratic_form(clauses):
    A = k_cnf_to_quadratic_form(clauses)
    det = determinant(A)
    if det == 0:
        return None
    return abs(det)

def communication_complexity_rank_variance(clauses):
    n = max(max(clause) for clause in clauses)
    rank = sum(1 for _ in range(n) if any(var in clause for clause in clauses))
    return rank ** 2

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            clauses = generate_k_cnf(n, n // 2)
            minimal_order_value = minimal_order_quadratic_form(clauses)
            if minimal_order_value is None:
                continue
            variance_rank_value = communication_complexity_rank_variance(clauses)
            results.append((minimal_order_value, variance_rank_value))
    if not results:
        return {
            "metric_name": "MinimalOrder",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 40,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }
    mean_diff = sum(b - a for a, b in results) / len(results)
    corr_coeff = sum((a - mean_a) * (b - mean_b) for a, b in results) / math.sqrt(sum((a - mean_a) ** 2 for a, b in results))
    return {
        "metric_name": "MinimalOrder",
        "metric_value": mean_diff,
        "instances_tested": len(results),
        "n_max": 40,
        "conjecture_holds": corr_coeff >= 0.8 and abs(mean_diff) <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
results.append(result)
mean_diff = sum(r[\"metric_value\"] for r in results if r[\"instances_tested\"] > 0) / len(results)
support_fraction = sum(1 for r in results if r[\"conjecture_holds\"] / len(results)
if all(r[\"instances_tested\"] > 0 for r in results):
    print(f"RESULT: SUPPORTED mean={mean_diff} std=NOT_COMPUTABLE support_fraction={support_fr
else:
    print("RESULT: INCONCLUSIVE reason=not_enough_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36eea172.py", line 99, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36eea172.py", line 72, in run_trial
    variance_rank_value = communication_complexity_rank_variance(clauses)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36eea172.py", line 60, in communication
    rank = sum(1 for _ in range(n) if any(var in clause for clause in clauses))
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36eea172.py", line 60, in <genexpr>
    rank = sum(1 for _ in range(n) if any(var in clause for clause in clauses))
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36eea172.py", line 60, in <genexpr>
    rank = sum(1 for _ in range(n) if any(var in clause for clause in clauses))
        ^^^

```

NameError: name 'var' is not defined. Did you mean: 'vars'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented it from producing any data to evaluate the conjecture. | next: Debug the error in the test code and rerun the test with a correct variable name.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14174
2	propose	ollama_remote	glm4:latest	0	0	12785
3	preregistration	ollama_remote	glm4:latest	0	0	9537
4	novelty	ollama_remote	glm4:latest	0	0	10606
5	novelty	ollama_remote	glm4:latest	0	0	9618
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15603
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10807
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18932
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13025
10	judge	ollama_remote	glm4:latest	0	0	17793

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132880 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/e4cad27d697b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e4cad27d697b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e4cad27d697b.tar.gz` (if generated)